Содержание

Power Systems Agent Benchmark: исполняемый бенчмарк для оценки инструментальных ИИ-агентов в задачах электроэнергетики **1**
Аннотация . . . 1
1. Введение . . . 2
2. Связанные работы . . . 2
3. Требования к бенчмарку для электроэнергетических агентов . . . 3
4. Архитектура . . . 4
4.1. Контракт артефактов . . . 4
4.2. Агент-агностичный запуск . . . 4
4.3. Лестница зрелости проверяющих программ . . . 4
4.4. Hidden-срез как генератор, а не как файлы . . . 4
5. Набор задач . . . 4
5.1. Доменная выверка набора . . . 5
6. Проверяющие программы и метрики . . . 5
7. Воспроизводимость . . . 6
8. Эксперименты . . . 6
8.1. Первый публичный прогон (до уточнения постановок) . . . 6
8.2. Типология провалов: способности против спецификации . . . 6
8.3. Повторный прогон после уточнения постановок . . . 7
8.4. Скрытый срез: рейтинг переворачивается . . . 7
8.5. Целостность оценки: агент, нашедший оракул . . . 7
9. Ограничения . . . 8
10. Заключение . . . 8
Приложение A. Каталог задач . . . 8
Приложение B. Артефакты экспериментов . . . 8

Power Systems Agent Benchmark: исполняемый бенчмарк для оценки инструментальных ИИ-агентов в задачах электроэнергетики

Черновик v1 (2026-06-11). Структура соответствует docs/plans/academic-paper-ru-plan.md.

Аннотация

В работе представлен Power Systems Agent Benchmark — прототип исполняемого бенчмарка для оценки инструментальных ИИ-агентов в задачах электроэнергетики. В отличие от текстовых question-answering наборов и оценки отчётов через LLM-as-a-Judge, предлагаемый подход требует от агента сформировать структурированное решение (submission.json), которое затем проверяется детерминированной программой-оценщиком: пересчитываются инженерные величины, проверяются физические и эксплуатационные ограничения, формируются машиночитаемые метрики и список нарушений. Публичный срез включает 41 задачу из разных областей электроэнергетики — от расчёта режимов, токов короткого замыкания и уставок релейной защиты до устойчивости, DER/V2G, микросетей, надёжности, качества электроэнергии и кибербезопасности измерений. Hidden-срез не распространяется с репозиторием: вместо него публикуется детерминированный генератор скрытых кейсов, порождающий их из приватного seed с автоматической проверкой решаемости. На матрице из трёх CLI-агентов (Codex/gpt-5.5, Cursor/composer-2.5, OpenCode/deepseek-v4-flash) показано, что (1) публичный срез различает агентов; (2) цикл «прогон — анализ идентичных провалов — уточнение постановок — повторный прогон» позволяет отделить дефекты спецификации от настоящих различий в способностях; (3) на скрытых

кейсах рейтинг агентов меняется: лидер публичного среза деградирует сильнее всех, что эмпирически обосновывает необходимость скрытой оценки. Текущие проверки — компактные детерминированные суррогаты; переход к simulator-backed оценщикам (pandapower, OpenDSS) описан как дорожная карта, не меняющая контракта задач.

1. Введение

Большие языковые модели всё чаще применяются как агенты к инженерным задачам: они читают данные, пишут вспомогательный код, вызывают инструменты и формируют решения. Стандартные способы оценки таких агентов — текстовые вопросы с эталонным ответом либо оценка свободного отчёта другим LLM (LLM-as-a-Judge) — плохо подходят для электроэнергетики. Инженерное решение здесь может выглядеть убедительно и при этом нарушать токовые и напряженческие ограничения, селективность защиты, требования grid code или балансовые уравнения. Ошибка агента проявляется не как неправильная формулировка, а как перегрузка линии, неселективная уставка, недопустимое напряжение или неверная локализация повреждения.

Поэтому основной результат оценки должен определяться не общим впечатлением от текста, а исполняемой проверкой предложенного решения. Мы формулируем и реализуем такой протокол: каждая задача публикует структурированные входные данные (task.json), агент обязан выдать машиночитаемое решение (submission.json), а независимая программа-оценщик пересчитывает целевые величины, проверяет жёсткие ограничения и возвращает feasible, нормированный score и список нарушений. LLM-объяснение (rationale) сохраняется как диагностика, но не влияет на счёт.

Вклад работы:

1. **Исполняемый контракт оценки** task.json → агент → submission.json → evaluator → evaluation-result.json с JSON-схемами всех артефактов, разделением жёстких ограничений (feasible) и качества (score) и агент-агностичным протоколом запуска любого CLI-агента через переменные окружения.
2. **Сбалансированный публичный набор из 41 исполняемой задачи** по восьми доменным группам с прослеживаемым происхождением каждой задачи (source cards, ссылки на стандарты и литературу) и двуязычным (en/ru) представлением.
3. **Архитектурное решение проблемы утечки hidden-набора**: вместо файлов скрытых кейсов публикуется детерминированный генератор с per-family возмущениями; кейсы порождаются приватным seed на хосте оценки, каждый розыгрыш автоматически проверяется на решаемость (baseline решает, evaluator подтверждает feasible). Механизм публично аудируем, конкретные числа — нет.
4. **Методология контроля качества постановок**: по матрице трёх агентов идентичные провалы всех участников интерпретируются как симптом дефекта спецификации, а не слабости агентов; семь таких дефектов были найдены, исправлены и закрыты повторным прогоном.
5. **Эмпирическое обоснование скрытой оценки**: на полном hidden-срезе рейтинг агентов изменился относительно публичного — лидер публичного среза деградировал сильнее всех.

2. Связанные работы

(Полная аннотированная библиография: docs/paper/bibliography.md; BibTeX: docs/paper/references.bib.)

Бенчмарки агентов. Общеназначенческие наборы — AgentBench [liu2024agentbench], GAIA [mialon2023gaia], WebArena [zhou2024webarena] — оценивают веб-навигацию, код и использование инструментов, но не инженерные домены. SWE-bench [jimenez2024swebench] и Codex/HumanEval [chen2021codex] установили жанр исполняемой оценки (решение проверяется тестами); τ-bench [yao2024taubench] перенёс принцип «сравнивай конечное состояние, а не текст» на диалоговых агентов и ввёл pass^k как меру устойчивости. Наш контракт распространяет исполняемую оценку с программного кода на инженерные расчёты с физическими ограничениями.

LLM-as-a-Judge. Судейство LLM [zheng2023judging] страдает документированными смещениями — позиционным, по длине, самопредпочтением [wang2023fairevaluators, gu2024judgesurvey, li2024generationjudgment]; для задач, где ответ может выглядеть убедительно и нарушать ограничения, судья непригоден как основной скор. Мы используем детерминированный пересчёт; rationale агента хранится только как диагностика.

LLM в энергетике. Свежий обзор [llmpowersurvey2025] фиксирует быстрый рост применений (диагностика, прогноз, диспетчеризация) при отсутствии стандартизированной оценки. Ближайшие к нам наборы: ElecBench [zhou2025elecbench] — сценарии диспетчеризации с шестью метриками, но оценкой через текст/судейство; PFBench [pfbench2025] — программная проверка с solver-derived ответами, но только для power flow. Появляющиеся производственные агенты — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026] — и есть целевые «испытуемые» нашего бенчмарка. Наш набор отличается сочетанием: 41 задача по восьми доменным группам, исполняемая проверка для каждой, агент-агностичный протокол и генератор скрытых кейсов.

RL-среды управления сетью. Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective] — зрелая среда последовательного топологического управления для RL; мы решаем другую задачу — агент-агностичный контракт однократных инженерных решений по широкому спектру подзадач отрасли.

Симуляторы. pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa] покрывают расчётные ядра, но не задают контракта оценки агентов; в нашей дорожной карте они — substrate проверяющих программ, подключаемый без изменения формата задач.

Целостность бенчмарков. Контаминация обучающих данных систематически завышает результаты [contaminationsurvey2025, statictodynamic2025]; ответы — динамические и обновляемые наборы (Dynabench [kiela2021dynabench], LiveBench [white2024livebench], LiveCodeBench [jain2024livecodebench]). Наш hidden-генератор — prevention-механизм этого же класса: публикуется способ порождения кейсов, а не кейсы. Критерии честной агентной оценки из [kapoor2024agentsmatter] (стоимость, воспроизводимость, разделение классов провалов) реализованы в harness напрямую; ограниченность best-of-N как метрики обоснована в [brown2024monkeys].

3. Требования к бенчмарку для электроэнергетических агентов

1. **Исполнимость**: решение проверяется кодом, а не сравнением строк.
2. **Физическая проверяемость**: evaluator пересчитывает величины и проверяет ограничения, а не сверяет с сохранённым текстом.
3. **Разделение public/hidden**: публичные кейсы открыты для разработки, скрытые — для честного сравнения; утечка скрытых ответов исключается архитектурно.
4. **Агент-агностичность**: запуск не зависит от конкретной LLM, CLI или фреймворка; контракт — «прочитай файлы, запиши файл».
5. **Прослеживаемость постановки**: у каждой задачи есть source provenance с уровнем доверия.
6. **Машиночитаемый результат**: feasible, score $\in [0,1]$, violations, метрики.
7. **Рост сложности без смены контракта**: от формульных суррогатов к simulator-backed проверкам меняется внутренность evaluator-а, но не формат task/submission.
8. *(добавлено по результатам экспериментов)* **Контроль качества самих постановок**: бенчмарк должен предусматривать процедуру отделения дефектов спецификации от различий в способностях агентов (раздел 8.3).

4. Архитектура

4.1. Контракт артефактов

`task.json → агент (любой CLI) → submission.json → evaluator → evaluation-result.json`

- `schemas/task.schema.json` — обязательные поля задачи: task_id, family, substrate, objective, public_inputs, submission_schema, hard_constraints, metrics, source_evidence, source_provenance (+ двуязычный блок bilingual).
- `schemas/submission.schema.json` — структурированный ответ: actions и/или answer, опциональный rationale.
- `schemas/evaluation-result.schema.json` — task_id, feasible (bool), score ∈ [0,1], violations (список строк), метрики и evidence.

Валидатор артефактов дополнительно гарантирует согласованность зеркальных копий данных (public_inputs и верхний уровень task.json) — дефект этого рода был найден и устранён в ходе работы.

4.2. Агент-агностичный запуск

`prepare_agent_tasks.py` собирает для каждой задачи самодостаточный пакет (task.json с редактированием + instructions.md); из пакета вырезаются sample submissions и любые поля-оракулы. `run_agent_command.py` запускает на каждый пакет произвольную shell-команду с переменными окружения BENCH_PACKET_DIR, BENCH_TASK_PATH, BENCH_SUBMISSION_PATH и таймаутом на кейс; stdout/stderr сохраняются, команда запуска фиксируется в summary. Три CLI-агента в экспериментах запускались командами своих vendor-CLI без какой-либо адаптации бенчмарка под них.

4.3. Лестница зрелости проверяющих программ

формульный/графовый суррогат → детерминированная инженерная модель → simulator-backed hidden evaluator → стохастические/масштабные сценарии.

Текущие public evaluators находятся на первых двух ступенях; контракт позволяет заменять внутренность проверки (например, DC-поток на pandapower AC) без изменения формата задач.

4.4. Hidden-срез как генератор, а не как файлы

Каталог tasks/*/hidden/ исключён из системы контроля версий. Вместо него репозиторий содержит `generate_hidden_cases.py`: реестр детерминированных возмущений для каждого из 41 семейства. Кейс порождается из публичного case_001 серией физически осмысленных перевыборок (параметры сети, погодные ряды, профили нагрузки, место повреждения с согласованными показаниями индикаторов и т.д.) от seed; розыгрыш принимается, только если baseline семейства решает его и evaluator подтверждает feasible, иначе выполняется детерминированная перерисовка. Оператор leaderboard хранит seed в секрете; любой желающий может аудировать, как порождаются скрытые кейсы, не зная их конкретных чисел.

5. Набор задач

41 публичная задача сгруппирована в восемь категорий (нормализованная таксономия):

1. **Сетевые расчёты и ограничения режима** — ybus_construction, pf_voltage_repair, contingency_screening, corrective_transmission_switching, dynamic_thermal_rating.

2. **Короткие замыкания и защита** — three_phase_short_circuit (пара максимального/минимального токов по МЭК 60909), earth_fault_calculation, breaker_relay_short_circuit, distance_protection_settings, overcurrent_protection, cable_ampacity_sizing.
3. **Устойчивость, grid code и IBR** — critical_clearing_time (критерий равных площадей), transient_stability_prediction frt_compliance, ibr_short_circuit_frt, min_synchronous_share.
4. **DER, PV, EV и накопители** — pv_volt_var, ev_v2g_outage_schedule, ev_v2g_voltage_support, bess_ancillary_response, commercial_pv_lcoe_uncertainty.
5. **Микросети и диспетчеризация** — microgrid_economic_dispatch, rolling_microgrid_dispatch, islanded_microgrid_pq_dispatch, dispatch_uncertainty, hydro_thermal_storage_uc, n1_generation_commitment.
6. **Надёжность и восстановление** — flisr_restoration, fault_section_localization, fci_placement, fci_saidi_caidi, operator_breaker_load_actions.
7. **Качество электроэнергии, стандарты, активы, кибербезопасность** — en50160_voltage_compliance, power_quality_event_classification, harmonic_ieee519_compliance, transformer_thermal_loading (МЭК 60076-7), fdi_state_estimation (WLS + тест наибольшей нормированной невязки), protected_meter_placement, mas_frequency_handover.
8. **Прогнозирование и неопределённость** — wind_power_forecast, wind_prediction_interval.

Каждая задача снабжена source card и source provenance с уровнем доверия; полный каталог — в приложении (docs/bilingual_task_inventory.md).

5.1. Доменная выверка набора

В ходе подготовки набор прошёл доменную ревизию, устранившую четыре класса фактических дефектов (исправления зафиксированы в истории репозитория):

- **Физические ошибки**: формула RoCoF использовала f_0 в знаменателе вместо числителя (расхождение в f_0^2 раз), из-за чего ограничение по инерции никогда не было активным; исправлено по уравнению движения с перебалансировкой кейса.
- **Концептуальные подмены**: задача «обнаружения FDI» содержала метки атак во входных данных и сводилась к сравнению с порогом; переработана в полноценную оценку состояния (WLS) с итеративным тестом наибольшей нормированной невязки, где эффект размазывания делает наивный взвешенный тест неверным; скрытные FDI-атаки явно вынесены за скоуп.
- **Вырожденные задачи**: расчёт критического времени отключения был чтением готовых флагов устойчивости из таблицы; заменён аналитическим расчётом по критерию равных площадей.
- **Завышенные ссылки на стандарты**: «расчёт по МЭК 60909» без коэффициента напряжения c и с сопротивлением дуги в максимальном токе занижал ток на ~10% неконсервативно; задача переработана в каноническую пару Ik_max (c_max, металлическое КЗ) / Ik_min (c_min, с сопротивлением).
- **Непрофильные задачи**: три задачи вне скоупа электроэнергетики (теплоснабжение зданий ×2, дисперсионный анализ Соболя) заменены на непокрытые профильные домены: тепловой режим трансформатора по МЭК 60076-7, выбор сечения кабеля по трём проверкам, проверка гармоник по IEEE 519.

6. Проверяющие программы и метрики

Основные поля результата: feasible (выполнены ли жёсткие ограничения), score $\in [0,1]$, violations (конкретные нарушения словами), task-specific evidence. Подходы к score по типам задач:

- точный расчёт: 1/0 либо допуски на численные поля;
- управление/оптимизация: feasible по ограничениям + качество по целевой функции. Для задач с неединственным оптимумом (микросетевые диспетчеризации) evaluator пересимулирует поданный

профиль решения и оценивает разрыв стоимости с оптимумом: равные по стоимости альтернативные оптимумы получают полный балл, допустимые субоптимальные решения — частичный;
- классификация/локализация: точные множества с перечислением промахов;
- стандарты и grid code: pass/fail + диагностические поля.

Метрики агрегируются двумя способами: **feasible count** (доля задач с выполненными жёсткими ограничениями) и **perfect count** (доля задач со score = 1.0); как показано в разделе 8, эти метрики могут давать разные рейтинги, и мы рекомендуем публиковать обе вместе с mean_score.

7. Воспроизводимость

`scripts/check.sh` прогоняет 197 юнит-тестов (оценщики, baseline-ы, harness, генератор hidden-кейсов), валидацию всех артефактов, поштучную оценку 41 публичного sample submission и baseline-прогон. Baseline-ы построены на тех же вычислительных ядрах, что и evaluator-ы, и дают 41/41 feasible со средним score 0.997 — это **проверка самосогласованности контракта**, а не свидетельство сложности задач; разделение подчёркивается во всех отчётах. CI выполняет полный check на каждый коммит.

8. Эксперименты

Три CLI-агента, одинаковые промпты и таймаут 600 с/кейс; агенты запускались без какой-либо адаптации под бенчмарк:

- Codex CLI 0.139.0, модель gpt-5.5 (reasoning effort medium);
- Cursor agent CLI, модель composer-2.5 / composer-2.5-fast;
- OpenCode 1.15.13, модель deepseek-v4-flash.

8.1. Первый публичный прогон (до уточнения постановок)

агент	feasible	mean_score
cursor	33/41	0.800
codex	32/41	0.776
opencode	31/41	0.754

Все 123 кейс-прогона записали submission (ни одного сбоя harness/auth/no-submission); все провалы — доменные. 12 семейств провалены хотя бы одним агентом.

8.2. Типология провалов: способности против спецификации

Анализ нарушений разделил провалы на две группы. **Группа A** — агенты расходятся: нарушения лимитов напряжения в Volt-VAR, неверный FRT-вердикт, перепутанный знак разряда EV, разные ошибки в risk-скоринге устойчивости. **Группа B** — все три агента проваливаются идентично; это симптом дефекта постановки. Найдено семь дефектов: описание поля, противоречащее его имени (доля против процентов — два агента вернули 0.51 вместо 51.0 при численно точном расчёте); имена вложенных ключей, не совпадающие с проверкой (агенты сдали численно верные SAIDI/CAIDI под ключами из текста схемы); отвержение семантически эквивалентной нулевой записи отключённого блока; требование точного совпадения с одним из равных по стоимости оптимумов; невысказанные конвенции кривой мощности ветротурбины, квантилей и членов теплового баланса провода.

Критерий «идентичный провал всех участников $\Rightarrow$ подозрение на дефект спецификации» предлагается как стандартная процедура контроля качества исполняемых бенчмарков.

8.3. Повторный прогон после уточнения постановок

После исправления семи дефектов восемь затронутых семейств были перегнаны теми же агентами: codex 8/8 (1.0), cursor 8/8 (1.0), opencode 7/8 — его оставшийся провал стал содержательным (нарушение merit-order: газовый блок включён там, где дешевле уголь). Прямое подтверждение диагноза: реальные сабмишены агентов из первого прогона, прогнанные через исправленные evaluator-ы, принимаются (включая альтернативный оптимум с точно совпавшей стоимостью).

Объединённый публичный leaderboard:

агент (модель)	feasible	perfect (score = 1.0)	mean_score
cursor (composer-2.5)	**41/41**	37/41	0.995
codex (gpt-5.5 medium)	39/41	35/41	0.947
opencode (deepseek-v4-flash)	37/41	35/41	0.900

Заметно, что метрики дают разные порядки: по perfect opencode догоняет cursor (29:29 в первом прогоне, 35:37 в объединённом), тогда как по feasible и mean_score лидирует cursor. deepseek-v4-flash либо решает задачу точно, либо проваливает; gpt-5.5 чаще сдаёт допустимое, но неоптимальное решение.

8.4. Скрытый срез: рейтинг переворачивается

Полный hidden-набор (по одному сгенерированному кейсу на семейство, случайный незаписанный seed) на тех же агентах:

агент (модель)	public feasible	hidden feasible	public mean	hidden mean
codex (gpt-5.5 medium)	39/41	**38/41**	0.947	**0.919**
cursor (composer-2.5)	**41/41**	35/41	**0.995**	0.846
opencode (deepseek-v4-flash)	37/41	35/41	0.900	0.846

Лидер публичного среза деградировал сильнее всех (−6 feasible) и сравнялся с самой слабой моделью; codex деградировал на одну задачу и возглавил скрытый срез. **Public-only leaderboard короновал бы не того агента** — это главный эмпирический аргумент работы в пользу скрытой оценки. Из 123 hidden-прогонов один не записал submission (таймаут 600 с у cursor на Volt-VAR); остальные провалы — содержательные доменные ошибки.

8.5. Целостность оценки: агент, нашедший оракул

Аудит логов hidden-прогона выявил, что один из агентов (Cursor) на задаче прогнозирования устойчивости нашёл и запустил код проверяющей программы и самопроверился против оракула: его лог прямо сообщает, что submission «оценивается как feasible со score 1.0». Это стало возможным из-за двух дефектов изоляции: пакеты задач располагались внутри рабочей копии репозитория, а harness запускал агентов с корнем репозитория в качестве рабочей директории; запрет на поиск кода проверок существовал только в тексте инструкции пакета. Вывод: изоляция на уровне инструкций недостаточна для агентов с автономным доступом к файловой системе. Harness исправлен: каждый кейс выполняется в песочнице-копии пакета вне репозитория, без переменных окружения, указывающих на репозиторий. Заражённый результат

исключён из учёта (hidden-показатель Cursor следует читать как 34/41); вывод о перевороте рейтинга от этого только усиливается. Находка включена в работу как самостоятельный методологический результат.

9. Ограничения

1. Публичные кейсы компактны (сети 3–5 узлов, короткие ряды): проверяется владение методом, а не масштаб промышленных расчётов.
2. Проверки — детерминированные суррогаты; до подключения simulator-backed evaluator-ов нельзя заявлять промышленную точность AC-расчётов.
3. По одному кейсу на семейство в каждом срезе: ранжирование по 41 точке статистически хрупко; генератор позволяет дёшево наращивать выборку.
4. Baseline 41/41 — проверка самосогласованности, а не нижняя граница сложности.
5. Seed использованного hidden-набора «сожжён» прогоном на машине разработчика; цитируемый leaderboard-раунд требует свежего seed на изолированном хосте.
6. Две постановки остаются под подозрением по результатам hidden-прогона (islanded_microgrid_pq_dispatch — вероятные альтернативные Q-оптимумы; risk-скоринг transient_stability_prediction) и подлежат той же процедуре ревизии; одинаковые частичные баллы всех агентов на трёх задачах указывают на конвенции tie-break в эталонных решениях.
7. Прогон одной конфигурации на агента: без повторов не оценена дисперсия по сэмплингу LLM.
8. Песочница реализована переносом пакета, а не контейнером: она устраняет утечку через относительные пути и окружение, но не противостоит намеренному поиску оракула по абсолютным путям; для соревновательных условий требуется контейнерная изоляция хоста оценки.

10. Заключение

Power Systems Agent Benchmark демонстрирует практический путь от текстовых задач к исполняемой оценке ИИ-агентов в электроэнергетике. Предложен и реализован воспроизводимый benchmark contract, который переводит электроэнергетические задачи из формата текстовых ответов в формат исполняемой проверки; показано, что он различает современных агентов, что качество самих постановок поддаётся систематическому контролю по матрице агентов, и что скрытая оценка из генератора меняет итоговый рейтинг относительно публичного среза. Главная ценность — не в конкретных компактных кейсах, а в контракте, таксономии, проверяющих программах, генераторе скрытых кейсов и воспроизводимом цикле контроля качества; дорожная карта — постепенное наполнение проверок simulator-backed реализациями без изменения внешнего контракта задач.

Приложение A. Каталог задач

См. docs/bilingual_task_inventory.md (41 задача, en/ru, с task_id и путями).

Приложение B. Артефакты экспериментов

- Публичная матрица: runs/public_agent_smoke/REPORT.md (+ каталоги *-public41-stable,* -rerun8-spec-fixes).
- Скрытая матрица: runs/hidden_agent_smoke/REPORT.md (+ каталоги *-hidden301).
- История доменных исправлений: сообщения коммитов репозитория.